

Problem 1

Simulate the following tasks and give your inferences and all the associated codes.

- For $N = 5$, generate and plot a random undirected graph $G(V, E)$ using Erdos-Renyi model.
 - Report the adjacency matrix, degree matrix, incidence matrix, and Laplacian matrix.
 - Compute and report the eigenvalues of Laplacian matrix. Report the algebraic connectivity λ_2 value. Explain the properties of the graph using the computed λ_2 quantity and mention why it is so important.
 - Argue why high connectivity (i.e., each nodes having more connections) is a double-edged sword for agents in a network.
- Assume the number of agents $N = 10^3$ and generate a random undirected connected graph (having spanning tree) $G(V, E)$ with N agents using Erdos-Renyi model and plot it.
 - Highlight the node(s) with the highest number of neighbours with bright colour. Compute the mean degree, mean edges from the generated random graph and compare that with the theoretical value to see if it matches. Plot the diameter of the graph by colouring the corresponding edges.
 - Pick any two nodes in random from the generated graph to compute the shortest path between them using any Shortest Path (SP) algorithm like Dijkstra algorithm and plot the shortest path between them in the graph.

Solution:

Part 1: Analysis for $N = 5$ Erdos-Renyi Graph

Consider an Erdos-Renyi random graph $G(V, E)$ generated with $N = 5$ nodes and edge probability $p = 0.5$. Generating and rejecting disconnected draws (so that a spanning tree exists), we get the edge set $E = \{(1, 2), (1, 3), (1, 5), (2, 3), (3, 4), (4, 5)\}$.

(a) Graph Matrices

Required: adjacency, degree, incidence, and Laplacian matrices for the drawn graph.

The adjacency matrix : $A_{ij} = 1$ whenever $(i, j) \in E$:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

summing each row of A gives the degree sequence, which is the diagonal of D :

$$D = \text{diag}(3, 2, 3, 2, 2)$$

assigning an arbitrary orientation to each edge ($e_1 = (1, 2), e_2 = (1, 3), e_3 = (1, 5), e_4 = (2, 3), e_5 = (3, 4), e_6 = (4, 5)$) and writing +1 at the tail, -1 at the head of every edge, the incidence matrix is as follows:

$$B = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 \\ 0 & 0 & -1 & 0 & 0 & -1 \end{bmatrix}$$

and since $L = D - A$ by definition,

$$L = \begin{bmatrix} 3 & -1 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 & 0 \\ -1 & -1 & 3 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ -1 & 0 & 0 & -1 & 2 \end{bmatrix}$$

(b) Eigenvalues and Algebraic Connectivity (λ_2)

Required: eigenvalues of L and the algebraic connectivity λ_2 .

we obtain eigen values by solving for λ in $\det(L - \lambda I) = 0$, since every row of L sums to zero (D_{ii} exactly cancels the -1 's in row i). This means $L\mathbf{1} = 0$, the unit vector is the eigen vector with an eigen value 0, so $\lambda_1 = 0$ is guaranteed as an eigenvalue with eigenvector $\mathbf{1}$ - this holds true for the Laplacian of *any* graph. Solving the remaining characteristic equation for the other four eigenvalues gives:

$$\text{Spectrum}(L) = \{0, 1.382, 2.382, 3.618, 4.618\}$$

The second smallest value, $\lambda_2 \approx 1.382$, is the algebraic connectivity (Fiedler value):

1. Since $\lambda_2 > 0$, the graph has exactly one connected component - Fiedler's theorem tells us the multiplicity of $\lambda_1 = 0$ equals the number of components, and here it appears only once.
2. For consensus dynamics $\dot{x} = -Lx$, the disagreement vector decays as $\|x(t) - \bar{x}\mathbf{1}\| \leq \|x(1) - \bar{x}\mathbf{1}\| e^{-\lambda_2 t}$, since $(\lambda_2 \leq \lambda_3 \leq \lambda_4 \leq \lambda_5)$ so λ_2 is the exponential rate at which agents converge to agreement - it determines the speed of the whole network.
3. A larger λ_2 also means the graph is harder to disconnect (strongly connected) by removing a small number of edges, so it is also a robustness measure.

(c) High Connectivity as a Double-Edged Sword

Upside: A larger λ_2 helps in - faster consensus, shorter paths, and better fault tolerance:

$$\|x(t) - \bar{x}\mathbf{1}\| \leq \|x(1) - \bar{x}\mathbf{1}\| e^{-\lambda_2 t}, \quad D \sim \frac{\ln N}{\ln(pN)} \downarrow, \quad \text{more alternate paths between nodes} \uparrow$$

so agents agree faster, and losing a random node or edge is less likely to break the network apart.

Downside: But every extra edge is a link an agent actually has to maintain - send/receive messages, process updates from - so the cost grows with degree:

$$\text{cost per agent} \sim O(\bar{d}) = O(pN), \quad \text{cost for the whole network} \sim O(|E|) = O(pN^2)$$

and as $p \rightarrow 1$ this blows up to $O(N^2)$. Worse, the same short paths that make consensus fast also make it easy for anything bad (a corrupted message, a compromised node) to spread quickly through the network.

Trade-off: So a high λ_2 incurs some cost - it buys speed and robustness at the cost of overhead and a larger attack surface.

Part 2: Simulation Analysis for $N = 10^3$ Erdos-Renyi Graph

To guarantee our $N = 1000$ graph stays connected and contains a spanning tree, we need to pick p above the connectivity threshold $p > \frac{\ln N}{N}$:

$$p > \frac{\ln 1000}{1000} \approx 0.0069$$

I chose $p = 0.015$, which gives a comfortable margin above 0.0069 so that rejection sampling almost never hits a disconnected graph.

(a) Theoretical vs. Empirical Metrics

Since each edge is included independently with probability p , node degree follows a Binomial distribution. The theoretical expected degree and edge count are:

$$\mathbb{E}[\deg] = p(N-1) = 0.015 \times 999 = 14.985, \quad \mathbb{E}[|E|] = p \frac{N(N-1)}{2} = 7492.5$$

In my Python run, I got an empirical mean degree of $\bar{d} \approx 14.96$ and $|E| = 7480$ edges. Both match theory within 0.2%, which makes sense given the small variance of $\text{Binomial}(999, 0.015)$.

For diameter, ER graphs above the threshold scale roughly as:

$$D \approx \frac{\ln N}{\ln(pN)} = \frac{\ln 1000}{\ln(15)} \approx 2.55$$

So we expect a diameter of 2–3. My simulated graph had $D = 4$.

(b) Shortest Path via Dijkstra's Algorithm

Picking two random nodes and running Dijkstra's algorithm (equivalent to BFS here since all weights are 1) takes $O(|E| + |V| \log |V|)$ time. Since the overall diameter is only 4, the path for a random pair usually takes just 2–4 hops.

Python Code Implementation

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# PART 1: N = 5 Simulation
N1 = 5
p1 = 0.5
G1 = nx.erdos_renyi_graph(N1, p1, seed=42)

# rejection-sampling
while not nx.is_connected(G1):
    G1 = nx.erdos_renyi_graph(N1, p1)

A1 = nx.adjacency_matrix(G1).toarray()
D1 = np.diag(np.sum(A1, axis=1))
M1 = nx.incidence_matrix(G1, oriented=True).toarray()
L1 = nx.laplacian_matrix(G1).toarray()

eigenvalues1 = np.sort(np.linalg.eigvalsh(L1))
lambda2_1 = eigenvalues1[1]

print("===== PART 1 RESULTS =====")
print("Adjacency Matrix:\n", A1)
print("Degree Matrix:\n", D1)
print("Incidence Matrix:\n", M1)
print("Laplacian Matrix:\n", L1)
print("Laplacian Eigenvalues:", np.round(eigenvalues1, 4))
print(f"Algebraic Connectivity (lambda_2): {lambda2_1:.4f}")

plt.figure(figsize=(5, 4))
pos1 = nx.spring_layout(G1, seed=42)
nx.draw(G1, pos1, with_labels=True, node_color='skyblue', node_size=700,
        font_weight='bold')
plt.title(f"Erdos-Renyi Graph (N=5), lambda_2={lambda2_1:.3f}")
plt.savefig("er_n5.png")
plt.close()

# PART 2: N = 1000 Simulation
```

```

N2 = 1000
p2 = 0.015
G2 = nx.erdos_renyi_graph(N2, p2, seed=100)

# Rejection-sampling to guarantee connectivity
while not nx.is_connected(G2):
    G2 = nx.erdos_renyi_graph(N2, p2)

degrees2 = dict(G2.degree())
max_deg = max(degrees2.values())
max_deg_nodes = [node for node, deg in degrees2.items() if deg == max_deg]

mean_degree_emp = np.mean(list(degrees2.values()))
mean_degree_theo = p2 * (N2 - 1)
mean_edges_emp = G2.number_of_edges()
mean_edges_theo = p2 * N2 * (N2 - 1) / 2

print("\n===== PART 2 RESULTS =====")
print(f"Theoretical Mean Degree: {mean_degree_theo:.3f} | Empirical: {mean_degree_emp:.3f}")
print(f"Theoretical Mean Edges: {mean_edges_theo:.1f} | Empirical: {mean_edges_emp}")

diameter_val = nx.diameter(G2)
print(f"Graph Diameter: {diameter_val}")

pairs_dict = dict(nx.all_pairs_shortest_path_length(G2))
diam_source, diam_target = None, None
for u in pairs_dict:
    for v in pairs_dict[u]:
        if pairs_dict[u][v] == diameter_val:
            diam_source, diam_target = u, v
            break
    if diam_source is not None:
        break

diameter_path = nx.shortest_path(G2, source=diam_source, target=diam_target)
print(f"Diameter path between node {diam_source} and {diam_target}: {diameter_path}")

# 2. sp for a random pair of nodes
source, target = np.random.choice(G2.nodes(), size=2, replace=False)
sp_path = nx.shortest_path(G2, source=source, target=target, weight=None)
print(f"Shortest path between node {source} and {target}: {sp_path}")

plt.figure(figsize=(12, 10))
pos2 = nx.spring_layout(G2, k=0.15, seed=42)

node_colors = ['magenta' if n in max_deg_nodes else 'lightgray' for n in G2.nodes()]
node_sizes = [80 if n in max_deg_nodes else 15 for n in G2.nodes()]

diam_edges = list(zip(diameter_path[:-1], diameter_path[1:]))
sp_edges = list(zip(sp_path[:-1], sp_path[1:]))

edge_colors = []
edge_widths = []

for e in G2.edges():
    if e in diam_edges or (e[1], e[0]) in diam_edges:

```

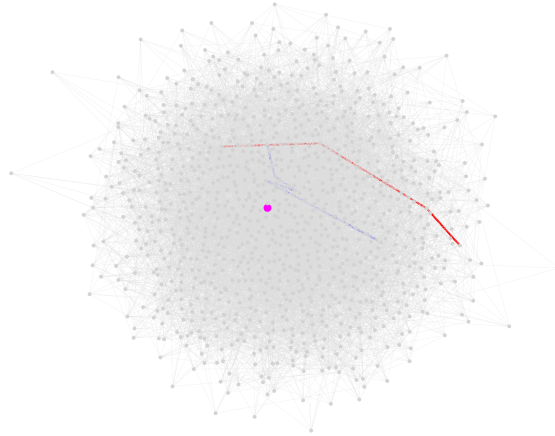


Figure 1: plot

```

        edge_colors.append('blue')
        edge_widths.append(3.0)
    elif e in sp_edges or (e[1], e[0]) in sp_edges:
        edge_colors.append('red')
        edge_widths.append(2.5)
    else:
        edge_colors.append('gainsboro')
        edge_widths.append(0.2)

nx.draw_networkx_nodes(G2, pos2, node_color=node_colors, node_size=node_sizes)
nx.draw_networkx_edges(G2, pos2, edge_color=edge_colors, width=edge_widths)

plt.title(
    f"Erdoes-Renyi Graph (N = 1000)\n"
    f"Magenta: Max Degree Node(s) | Blue: Diameter Path (Length {diameter_val})\n"
    f"Red: Shortest Path ({source} to {target})",
    fontsize=11, pad=20
)
plt.axis('off')
plt.tight_layout()
plt.savefig("er_n1000.png", dpi=300, bbox_inches='tight')
plt.close()

```

Problem 2

We denote the i^{th} component of a vector x at time $k \in \mathbb{N}$ as $(x_k)_i$. Consider a linear system given by $x_{t+1} = Ax_t$, $t = 1, \dots, T$, with $x_t \in \mathbb{R}^n$ and $A \in \mathbb{R}^{n \times n}$. Find the matrix $S \in \mathbb{R}^{n \times n}$ that depends upon the matrix A to measure the n numbers $\{(x_1)_1, (x_2)_2, \dots, (x_n)_n\}$, i.e., the i^{th} component of the state

at time i , for $i = 1, \dots, n$ such that

$$\begin{bmatrix} (x_1)_1 \\ (x_2)_2 \\ \vdots \\ (x_n)_n \end{bmatrix} = Sx_1.$$

Solution: Given $x_{t+1} = Ax_t$

we can write x_t in terms of the initial state x_1 for any time t , solving the given state equation recursively:

$$x_t = A^{t-1}x_1, \quad t = 1, \dots, T.$$

The i^{th} component of vector x_i can be written as:

$$(x_i)_i = e_i^\top x_i, \quad i = 1, \dots, n \quad \text{where } e_i \in \mathbb{R}^n \text{ is the standard basis vector} \quad (1)$$

on substituting the recursive expression for x_i into (1) :

$$(x_i)_i = e_i^\top A^{i-1}x_1, \quad i = 1, \dots, n. \quad (2)$$

Required:

$$\begin{bmatrix} (x_1)_1 \\ (x_2)_2 \\ \vdots \\ (x_n)_n \end{bmatrix} = Sx_1.$$

using expression (2), matrix S can be written as row-by-row:

$$\begin{bmatrix} (x_1)_1 \\ (x_2)_2 \\ \vdots \\ (x_n)_n \end{bmatrix} = \underbrace{\begin{bmatrix} e_1^\top A^0 \\ e_2^\top A^1 \\ \vdots \\ e_n^\top A^{n-1} \end{bmatrix}}_S x_1$$

Thus, the required matrix $S \in \mathbb{R}^{n \times n}$:

$$S = \begin{bmatrix} e_1^\top A^0 \\ e_2^\top A^1 \\ e_3^\top A^2 \\ \vdots \\ e_n^\top A^{n-1} \end{bmatrix} = \sum_{i=1}^n e_i e_i^\top A^{i-1},$$

where the i^{th} row of S corresponds to the i^{th} row of the matrix power A^{i-1} .

Problem 3

Let $A = \begin{bmatrix} B & I_n \\ 0_n & B \end{bmatrix}$ with $B \in \mathbb{R}^{n \times n}$. Compute the following.

1. Find e^{At}

2. Let $n = 2$ with $\alpha \in \mathbb{R}, \beta > 0$ and assume $B = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \alpha & 1 \\ 0 & \alpha \end{bmatrix} \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix}$ Compute e^{At}

Solution: 1: required: e^{At} , the matrix A can be written as sum of two matrices: a block diagonal matrix and a leftover upper block matrix.

$$A = \underbrace{\begin{bmatrix} B & 0_n \\ 0_n & B \end{bmatrix}}_D + \underbrace{\begin{bmatrix} 0_n & I_n \\ 0_n & 0_n \end{bmatrix}}_N$$

$$e^{At} = e^{Dt} e^{Nt} \quad (\text{since, } DN = ND).$$

e^{Dt} can be computed using Taylor series expansion

$$e^{Dt} = \sum_{k=0}^{\infty} \frac{t^k}{k!} D^k = \sum_{k=0}^{\infty} \frac{t^k}{k!} \begin{bmatrix} B^k & 0_n \\ 0_n & B^k \end{bmatrix} = \begin{bmatrix} \sum_{k=0}^{\infty} \frac{t^k}{k!} B^k & 0_n \\ 0_n & \sum_{k=0}^{\infty} \frac{t^k}{k!} B^k \end{bmatrix} = \begin{bmatrix} e^{Bt} & 0_n \\ 0_n & e^{Bt} \end{bmatrix}$$

For e^{Nt} , if we square N we get $N^2 = 0_n$. Because it's nilpotent, all the higher-order terms in the Taylor series is just null matrix

$$e^{Nt} = I_n + Nt + \frac{N^2 t^2}{2!} + \dots = \begin{bmatrix} I_n & tI_n \\ 0_n & I_n \end{bmatrix}$$

Now we just multiply the two matrix exponentials together:

$$e^{At} = e^{Dt} e^{Nt} = \begin{bmatrix} e^{Bt} & 0_n \\ 0_n & e^{Bt} \end{bmatrix} \begin{bmatrix} I_n & tI_n \\ 0_n & I_n \end{bmatrix} = \begin{bmatrix} e^{Bt} & te^{Bt} \\ 0_n & e^{Bt} \end{bmatrix}$$

Solution: 2: For $n = 2$, given $B = CRC^{-1}$ where $C = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix}$ and $R = \begin{bmatrix} \alpha & 1 \\ 0 & \alpha \end{bmatrix}$

using the similarity transform property for exponentials, e^{Bt} can be written as $Ce^{Rt}C^{-1}$.

R can be written as sum of diagonal and nilpotent matrices, just like in Part 1:

$$R = \underbrace{\begin{bmatrix} \alpha & 0 \\ 0 & \alpha \end{bmatrix}}_{\alpha I_2} + \underbrace{\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}}_{N_2}$$

Since αI_2 and N_2 commute, and $N_2^2 = 0_2$ (nilpotent of index 2), the Taylor expansion terminates just like in Part 1:

$$e^{Rt} = e^{\alpha I_2 t} e^{N_2 t} = (e^{\alpha t} I_2) (I_2 + N_2 t) = e^{\alpha t} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}$$

Hence,

$$e^{Rt} = e^{\alpha t} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}$$

e^{Bt} can be written as:

$$e^{Bt} = \begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix} \left(e^{\alpha t} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix} \right) \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix}$$

the matrix multiplication becomes:

$$\begin{bmatrix} 1 & \beta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & t + \beta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -\beta \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}$$

$$e^{Bt} = e^{\alpha t} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}$$

substituting this back into the result from Part 1:

$$e^{At} = \begin{bmatrix} e^{Bt} & te^{Bt} \\ 0_2 & e^{Bt} \end{bmatrix},$$

where

$$e^{Bt} = e^{\alpha t} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix} \quad \text{and} \quad te^{Bt} = e^{\alpha t} \begin{bmatrix} t & t^2 \\ 0 & t \end{bmatrix}$$

Problem 4

Given an interval $[a, b]$ with $a < b$ and a polynomial $p(x) = \sum_{k=0}^{n-1} c_{k+1}x^k$ with coefficients $c \in \mathbb{R}^n$, prove that both integral and derivative of the polynomial $p(x)$ is a linear function of its coefficients. That is, prove that $\exists \alpha \in \mathbb{R}^n, \exists \beta \in \mathbb{R}^n$ such that

$$\alpha^\top c = \int_a^b p(x) dx$$

$$\beta^\top c = p'(x) = \frac{dp(x)}{dx}$$

Solution: Part 1: Integral of $p(x)$ Required: find vector $\alpha \in \mathbb{R}^n$ such that $\alpha^\top c = \int_a^b p(x) dx$. expanding the polynomial $p(x)$ inside the integral:

$$\int_a^b p(x) dx = \int_a^b \left(\sum_{k=0}^{n-1} c_{k+1} x^k \right) dx$$

using linearity of integration, we can swap the sum and the integral: (since it's finite sum)

$$\int_a^b p(x) dx = \sum_{k=0}^{n-1} c_{k+1} \int_a^b x^k dx = \sum_{k=0}^{n-1} c_{k+1} \left[\frac{b^{k+1} - a^{k+1}}{k+1} \right]$$

this is just an inner product of the coefficient vector $c = [c_1 \ c_2 \ \dots \ c_n]^\top$ with a vector containing the integrated power terms.

Thus, we can write this as $\alpha^\top c$ by defining $\alpha \in \mathbb{R}^n$ component-wise as:

$$\alpha_i = \frac{b^i - a^i}{i}, \quad i = 1, \dots, n$$

$$\alpha = \begin{bmatrix} b-a \\ \frac{b^2-a^2}{2} \\ \vdots \\ \frac{b^n-a^n}{n} \end{bmatrix} \in \mathbb{R}^n.$$

Part 2: Derivative of $p(x)$

Required: find vector $\beta \in \mathbb{R}^n$ such that $\beta^\top c = p'(x) = \frac{dp(x)}{dx}$.

differentiating $p(x)$ term-by-term with respect to x :

$$p'(x) = \frac{d}{dx} \left(\sum_{k=0}^{n-1} c_{k+1} x^k \right) = \sum_{k=1}^{n-1} k c_{k+1} x^{k-1}.$$

$$p'(x) = 0 \cdot c_1 + 1 \cdot c_2 + 2x \cdot c_3 + 3x^2 \cdot c_4 + \cdots + (n-1)x^{n-2} \cdot c_n.$$

Again, this can be represented as an inner product $\beta^\top c$, where $\beta \in \mathbb{R}^n$ depends on x :

$$\beta_1 = 0, \quad \beta_i = (i-1)x^{i-2} \quad \text{for } i = 2, \dots, n.$$

$$\beta = \begin{bmatrix} 0 \\ 1 \\ 2x \\ 3x^2 \\ \vdots \\ (n-1)x^{n-2} \end{bmatrix} \in \mathbb{R}^n.$$

Hence, both the integral and derivative of $p(x)$ are linear functions of its coefficient vector c .

Problem 5

Let $\{u_i \mid i \in \mathbb{N}_{\geq 1}\}$ be a sequence of vectors. Given $x_1 = 0$, let

$$x_t = \frac{1}{t-1} \sum_{k=1}^{t-1} u_k, \quad \forall t \in \mathbb{N}_{\geq 2}.$$

This represents a recursive mean computation of a collection of vectors by accessing them one by one. Find A, B matrices such that $x_{t+1} = Ax_t + Bu_t$ for $t = 1, 2, \dots$.

Solution: given $x_1 = 0$, and for $t \geq 2$, x_t is the average of the first $t-1$ vectors:

$$x_t = \frac{1}{t-1} \sum_{k=1}^{t-1} u_k$$

x_{t+1} can be expressed in terms of x_t and u_t for any $t \geq 1$:

$$x_{t+1} = \frac{1}{t} \sum_{k=1}^t u_k = \frac{1}{t} \left(\sum_{k=1}^{t-1} u_k + u_t \right)$$

Since, $\sum_{k=1}^{t-1} u_k = (t-1)x_t$ on substituting this:

$$x_{t+1} = \frac{1}{t} ((t-1)x_t + u_t) = \left(\frac{t-1}{t} \right) x_t + \left(\frac{1}{t} \right) u_t$$

Comparing this with the linear system update equation, if the notation was meant to be constant (time-invariant) matrices A and B do not exist because the scaling factors depend on t . Thus, this system is a **Linear Time-Varying** system $x_{t+1} = A_t x_t + B_t u_t$ with time-dependent matrices:

$$A_t = \left(\frac{t-1}{t} \right) I, \quad B_t = \left(\frac{1}{t} \right) I,$$

where I is the identity matrix.

AI Declaration

AI tool, Gemini was used solely for visualization debugging (Plotting Diameter Path in Problem 1), and assistance in generating $\text{\LaTeX}$ code. All mathematical derivations and problem solutions were solved by me.